Uncertainty-Aware Multi-Agent Ransomware Detection via Decision MLP Fusion, Score-Space Augmentation, and Bayesian Triage Policy Optimisation

Henry Kabuye*, Biju Issac*, Senior Member, IEEE, Rahul Yumlembam*, Zahra Farzadpour*, and Jeyamohan Neera*, Member, IEEE

Abstract—Ransomware detection under operational constraints demands more than a binary verdict: it requires principled uncertainty estimation, heterogeneous evidence fusion, and uncertainty-driven triage that balances autonomous containment against analyst escalation. We present a three-agent framework addressing these requirements through four coupled contributions. First, a Semantic CNN on static PE embeddings, a Behavioural CNN on 16-dimensional hypervisor telemetry, and a Knowledge CNN on ATT&CK tactic embeddings each employ Monte Carlo Dropout to produce per-sample predictive probabilities and epistemic uncertainties. Second, a Decision MLP learns a calibrated aggregate risk score from the six-dimensional score pool, reducing Expected Calibration Error from 0.094 to 0.018; Conditional Tabular GAN (CTGAN) augmentation addresses score-space imbalance without retraining base agents. Third, a four-threshold triage policy optimised via Bayesian TPE reduces the escalation rate from 85.3% to 24.7% while raising balanced accuracy from 0.525 to 0.866. Fourth, a Peer Consultation mechanism mitigates knowledge collapse under concurrent benign–ransomware execution, and an Expert Feedback Loop with Elastic Weight Consolidation enables continual adaptation to novel families with negligible catastrophic forgetting (ΔAUPRC=0.001). Evaluated across four generalisation axes over six independent seeds on RanSMAP, RDset, and RansKnow, the framework achieves AUROC 0.996 on static PE detection, AUROC 0.961 on runtime behavioural detection, and AUROC 0.980 under temporal holdout. Under family holdout, Decision MLP AUROC falls to 0.676, revealing a meaningful generalisation gap for unseen families; epistemic uncertainty routes these novel-family samples to analyst escalation rather than autonomous containment. A Mondrian conformal prediction wrapper provides distribution-free coverage guarantees, auto-classifying 77% of traffic at 99% coverage.

Index Terms—Ransomware detection, multi-agent systems, uncertainty quantification, generative adversarial networks, continual learning, conformal prediction.

I. Introduction

Ransomware attacks impose escalating operational and financial costs on enterprises and critical infrastructure [1], [2]. State-of-the-art detectors—whether signature-based antivirus engines, endpoint detection and response (EDR) platforms, or machine-learning classifiers trained on file-level features—converge on a common architectural limitation: they produce a single scalar confidence score and map it to a binary verdict. When a novel variant or an obfuscated execution pattern degrades that score silently, no signal informs the operator that the verdict is unreliable.

We identify three structural gaps in the current state of practice:

1) No principled uncertainty. A softmax output from a deterministic neural network is a point estimate of class membership, not a probability over models. It cannot distinguish a confident correct prediction from a confident incorrect one.
2) Heuristic modality fusion. Combining static PE features with runtime telemetry via equal-weight averaging discards the per-modality reliability signal; a semantic agent operating on encrypted PE sections and a behavioural agent observing obfuscated I/O patterns should not carry equal weight.
3) Binary triage. An alert generated by every uncertain detection overwhelms analysts; equally, routing every uncertain detection to automatic containment risks business disruption.

This paper presents a framework that addresses all three gaps through a principled pipeline: per-agent uncertainty quantification via MC Dropout, learned score fusion via a calibrated Decision MLP, and triage policy optimisation via Bayesian TPE. The framework treats uncertainty as a control variable for deployment rather than a diagnostic afterthought.

Contributions

1) A heterogeneous three-agent architecture in which a Semantic CNN, a Behavioural CNN, and a Knowledge CNN each produce a predictive probability and an epistemic uncertainty via $T = 10$ MC Dropout forward passes.
2) A Decision MLP fusion mechanism with temperature calibration that replaces heuristic mean fusion, reducing ECE from 0.094 to 0.018, supported by

*Department of Computer and Information Sciences, Northumbria University, Newcastle, NE1 8ST, UK. E-mail: {kabuye-henry2017, bissac, rahul.yumlembam, zahra.farzadpour, jeyamohan.neera}@northumbria.ac.uk.

B. Issac is the corresponding author. E-mail: bissac@ieee.org.

CTGAN score-space augmentation to handle class imbalance at the fusion layer.
3) A formal triage policy over $(\hat{p}, \sigma_{\max})$ space with four jointly optimised thresholds via Bayesian TPE, reducing escalation rate from 85.3% to 24.7% while improving balanced accuracy from 0.525 to 0.866.
4) A Peer Consultation mechanism that prevents knowledge collapse under concurrent execution, and an Expert Feedback Loop with EWC enabling continual family adaptation without catastrophic forgetting.

The remainder of this paper is organised as follows. Section II reviews related work. Section III formalises the detection and triage problem. Section IV describes the three agents and MC Dropout uncertainty. Section V presents Decision MLP fusion and CTGAN augmentation. Section VI derives the triage policy and Bayesian optimisation. Section VII describes the Peer Consultation and Expert Feedback Loop. Section VIII covers conformal prediction. Section IX presents experimental evaluation. Section X discusses results and limitations. Section XI concludes.

II. Related Work

A. Behavioural Ransomware Detection

Hypervisor-level behavioural monitoring [3], [4] captures low-level I/O and memory access patterns before encryption artefacts surface in the filesystem, providing strong coverage of novel variants. I/O Request Packet (IRP) monitors [5] detect read–encrypt–write sequences characteristic of bulk encryption but remain vulnerable to slow-encrypt evasion that artificially suppresses throughput statistics. API-call-based classifiers [6], [7] construct fine-grained behavioural profiles from system-call sequences and achieve strong accuracy on closed-world benchmarks, yet require process-level instrumentation that incurs non-trivial overhead at scale. Survey work [8], [9] confirms that runtime behavioural methods are consistently more effective than static analysis against obfuscated or packed ransomware, but notes that all existing systems share a structural limitation: they produce a scalar confidence score and provide no mechanism for expressing per-sample epistemic uncertainty or routing uncertain detections to analyst escalation.

B. Uncertainty Quantification in Detection

Monte Carlo Dropout [10] re-activates dropout at test time, treating each stochastic forward pass as a draw from an approximate posterior over network weights; the variance of T outputs approximates epistemic uncertainty at the cost of $T\times$ the inference compute. Gawlikowski et al. [11] provide a systematic taxonomy of neural-network uncertainty methods across four families—ensemble, sampling, deterministic, and test-time augmentation—establishing that MC Dropout and deep ensembles are the most broadly applicable. Deep ensembles [12] independently trained on random initialisations achieve better marginal calibration than MC Dropout but require retaining M full model copies at inference. For distribution-free guarantees, conformal prediction [13] and its class-conditional Mondrian variant [14] bound coverage probability without parametric assumptions; Angelopoulos and Bates [15] generalise this to arbitrary risk functions. In the security domain, uncertainty estimation has been applied to network anomaly detection [16] and intrusion classification, but ransomware triage—where the cost of an autonomous containment error differs qualitatively from the cost of an escalation—has not previously been framed as an uncertainty-conditioned routing problem.

C. Multi-Agent and Ensemble Detection

Ensemble methods for malware classification [17], [18] aggregate multiple classifiers over a shared feature space, improving robustness through majority vote or average-score fusion. Multi-agent intrusion detection systems [19] distribute the detection task across specialised agents that monitor different network zones, combining alerts through a centralised coordinator. Multi-modal ransomware detection combining static PE features, API-call sequences, and file-system events has been explored [8], but typically with equal-weight or attention-based fusion that does not propagate per-modality reliability.

Our framework differs from all of the above in two structural respects. First, the three agents operate on structurally heterogeneous input spaces ($\mathbb{R}^{384}$ PE embeddings, $\mathbb{R}^{16}$ runtime telemetry, $\mathbb{R}^{23}$ ATT&CK vectors) that cannot be concatenated without discarding modality-specific structure. Second, each agent contributes not only a probability but an epistemic uncertainty, which the Decision MLP uses implicitly to down-weight unreliable modalities and which the orchestrator uses explicitly to route uncertain cases to analyst escalation rather than autonomous containment. Standard ensemble methods have no equivalent of this uncertainty-conditioned routing.

D. Generative Augmentation and Continual Learning

Class imbalance in detection datasets is routinely addressed by SMOTE [20] and Conditional Tabular GAN (CTGAN) [21], which have been applied to fraud and intrusion detection [22], [23], consistently outperforming SMOTE when the minority class is small and feature distributions are non-Gaussian. Our contribution applies CTGAN at the score space output of the multi-agent detector: operating on bounded $[0, 1]^6$ probability–uncertainty vectors constrains synthetic samples to the agents' output range, reducing the risk of generating implausible score-uncertainty combinations and eliminating raw telemetry reconstruction artefacts.

Catastrophic forgetting is the key barrier to continual adaptation: gradient updates for a new task systematically overwrite prior weight configurations [24]. Temporal evaluation frameworks [25], [26] and systematic studies [27] establish that all standard classifiers lose substantial accuracy within months of the training cutoff under concept drift. Elastic Weight Consolidation (EWC) [24]

mitigates this via a diagonal Fisher penalty that protects parameters important to retained tasks. Our Expert Feedback Loop applies EWC to the Decision MLP, achieving ΔAUPRC=0.001 on retained families after five correction iterations—enabling continual adaptation without retraining base agents.

E. Relationship to Prior Work

This paper extends the two-agent Agentic SABRE framework [28], which combined a Semantic CNN and a Behavioural CNN with MC Dropout uncertainty and a fixed triage policy. The present work makes four technically distinct contributions absent from the prior system: (1) a third Knowledge CNN agent on ATT&CK tactic embeddings, expanding the score pool from $[0,1]^4$ to $[0,1]^6$; (2) Bayesian TPE optimisation of four jointly tuned triage thresholds $(\tau_{\text{high}}, \tau, \kappa, \kappa_{\text{low}})$ replacing the fixed (τ=0.10, κ=0.10) pair; (3) CTGAN score-space augmentation at the fusion layer; and (4) EWC continual learning and Mondrian conformal prediction wrappers. All datasets, evaluation splits, experimental results, and figures are independent of [28].

III. Problem Formulation

A. Detection Task

Let $\mathcal{X} = \mathcal{X}_{\text{sem}} \times \mathcal{X}_{\text{beh}} \times \mathcal{X}_{\text{know}}$ denote the joint input space of three heterogeneous modalities, and let $\mathcal{Y} = \{0, 1\}$ denote the binary ransomware label. A sample is observed as a triple $\mathbf{x} = (\mathbf{x}_{\text{sem}}, \mathbf{x}_{\text{beh}}, \mathbf{x}_{\text{know}}) \in \mathcal{X}$, where

- $\mathbf{x}_{\text{sem}} \in \mathbb{R}^{384}$ is a sentence-transformer embedding of Portable Executable (PE) section names;
- $\mathbf{x}_{\text{beh}} \in \mathbb{R}^{16}$ is a 16-dimensional feature vector extracted from a $T_{\text{window}} = 10\,\text{s}$ telemetry window (see Section IV);
- $\mathbf{x}_{\text{know}} \in \mathbb{R}^{d}$ is an embedding of ATT&CK tactic profiles derived from threat-intelligence transcripts.

B. Triage Task

Rather than mapping $\mathbf{x}$ to a binary label, the triage task maps $\mathbf{x}$ to one of three actions $\mathcal{A}$ = {Auto-Contain, Escalate, Allow} according to a policy $\pi : (\hat{p}, \sigma_{\max}) \to \mathcal{A}$, where $\hat{p} \in [0,1]$ is the fused risk score and $\sigma_{\max} \in [0,1]$ is the maximum per-agent epistemic uncertainty.

Definition 1 (Triage Policy). Given thresholds $\Theta = (\tau_{\text{high}}, \tau, \kappa, \kappa_{\text{low}})$ with $0 < \tau \le \tau_{\text{high}} \le 1$ and $0 < \kappa_{\text{low}} \le \kappa \le 1$, the triage policy is:

$$\pi(\hat{p}, \sigma_{\max}; \Theta) = \begin{cases} \text{Auto-Contain} & \hat{p} \ge \tau_{\text{high}} \ \wedge \ \sigma_{\max} \le \kappa_{\text{low}}, \\ \text{Escalate} & \hat{p} \ge \tau \ \vee \ \sigma_{\max} > \kappa, \\ \text{Allow} & \text{otherwise.} \end{cases} \tag{1}$$

C. Optimisation Objective

Let bal_acc(Θ) denote the balanced accuracy of the joint detection–triage system under thresholds Θ, and let esc_rate(Θ) denote the fraction of samples routed to Escalate. The policy optimisation problem is:

$$\Theta^* = \arg\max_{\Theta \in \mathcal{F}} \ J(\Theta) \coloneqq \text{bal_acc}(\Theta) - \lambda\, \text{esc_rate}(\Theta), \tag{2}$$

where $\lambda \ge 0$ is an operational cost parameter encoding the relative burden of analyst escalation, and $\mathcal{F}$ is the feasibility region $\{\Theta : \tau \le \tau_{\text{high}},\ \kappa_{\text{low}} \le \kappa\}$.

Remark 1. The parameter λ is a deployment decision, not a modelling one. An organisation with constrained analyst capacity sets higher λ; one prioritising missed-detection avoidance sets $\lambda \approx 0$.

IV. Agent Architecture and Uncertainty Quantification

A. Monte Carlo Dropout as Approximate Bayesian Inference

Let $f_\theta : \mathcal{X}_i \to [0,1]$ denote a neural network with parameters θ trained on modality i. At inference, standard dropout is disabled, yielding a deterministic point estimate. MC Dropout [10] instead keeps dropout active at test time, treating each stochastic forward pass as a draw from an approximate posterior $q_\phi(\theta) \approx p(\theta \mid \mathcal{D})$ over network weights.

Formally, for T stochastic forward passes with dropout masks $\{\mathbf{z}_t\}_{t=1}^{T}$:

$$\hat{p}_i(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^{T} f_{\theta \odot \mathbf{z}_t}(\mathbf{x}), \tag{3}$$

$$\sigma_i(\mathbf{x}) = \sqrt{\frac{1}{T-1} \sum_{t=1}^{T} \left(f_{\theta \odot \mathbf{z}_t}(\mathbf{x}) - \hat{p}_i(\mathbf{x}) \right)^2}, \tag{4}$$

where $\odot$ denotes element-wise masking. The quantity σ_i captures epistemic uncertainty—reducible uncertainty arising from limited training data—as distinct from aleatoric uncertainty inherent to the data generating process [29].

Proposition 1 (Epistemic Signal). When a test sample $\mathbf{x}$ lies far from the training manifold of agent i (i.e., a novel ransomware family), we hypothesise that the variance $\sigma_i^2(\mathbf{x})$ tends to increase with distributional shift $d(\mathbf{x}, \mathcal{D}_{\text{train}})$ [10]. Approximate Bayesian methods do not provide a formal monotonicity guarantee in general; we verify this tendency empirically across family and hardware holdout splits (Supplementary Section S-VI).

We use $T = 10$ passes per agent, empirically verified to stabilise σ_i to within 1% of the $T = 50$ estimate (Section IX-N). Supplementary Section S-VI quantifies this stability and the associated latency trade-off.

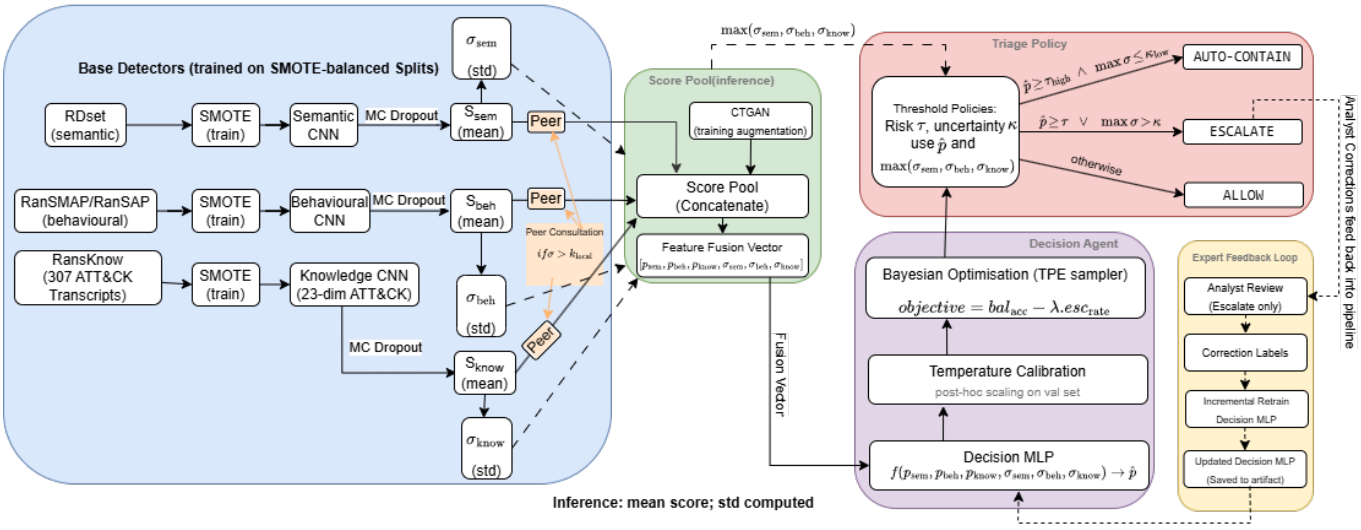


Fig. 1. End-to-end framework pipeline. Three specialised CNN agents (Semantic, Behavioural, Knowledge) each run T=10 MC Dropout forward passes to produce a probability–uncertainty pair $(\hat{p}_i, \sigma_i)$. Peer Consultation adjusts any agent whose σ_i exceeds its local threshold before the six-dimensional score pool is assembled. The Decision MLP fuses the pool into a calibrated risk score; Bayesian TPE derives the four-threshold triage policy that maps $(\hat{p}, \sigma_{\max})$ to Auto-Contain, Escalate, or Allow. Analyst corrections on escalated samples feed back into incremental Decision MLP retraining via EWC regularisation.

B. Semantic Agent

The Semantic Agent operates on static PE structure. Given a PE file, section names are concatenated and encoded via a frozen sentence transformer (all-MiniLM-L6-v2) to produce $\mathbf{x}_{\text{sem}} \in \mathbb{R}^{384}$. A Semantic CNN f_{sem} with MC Dropout maps this embedding to $(\hat{p}_{\text{sem}}, \sigma_{\text{sem}})$.

Training data: RDset [30], [31], comprising 138,047 PE files (41,123 ransomware; 96,924 benign) with SMOTE balancing applied at the feature level prior to training.

C. Behavioural Agent

The Behavioural Agent operates on runtime storage and memory access patterns collected via a thin BitVisor hypervisor [4]. Within each $T_{\text{window}} = 10\,\text{s}$ window we compute a $\mathbf{x}_{\text{beh}} \in \mathbb{R}^{16}$ feature vector partitioned as $\mathbf{x}_{\text{beh}} = (\chi_s, \chi_m)$, where:

Storage features $\chi_s \in \mathbb{R}^5$:

$$\chi_s = \Big(\bar{T}_{s,r},\ \bar{T}_{s,w},\ V_{s,r},\ V_{s,w},\ \bar{H}_{s,w}\Big), \tag{5}$$

where $\bar{T}_{s,r}$ and $\bar{T}_{s,w}$ are mean read/write throughputs [byte/s]; $V_{s,r}$ and $V_{s,w}$ are variances of accessed Logical Block Addresses (LBA); and $\bar{H}_{s,w}$ is mean Shannon entropy of written sectors, computed as $H(X) = -\sum_i p(x_i) \log_2 p(x_i)$.

Memory features $\chi_m \in \mathbb{R}^{11}$:

$$\chi_m = \Big(H_{m,w},\ H_{m,rw},\ C_{4K,r},\ C_{4K,w},\ C_{4K,rw},\ C_{4K,x},\ C_{2M,r},\ C_{2M,rw},\ C_{\text{MMIO},r},\ C_{\text{MMIO},w},\ V_{m,w}\Big), \tag{6}$$

where $H_{m,w}$ and $H_{m,rw}$ are Shannon entropies of written and read/written memory pages; $C_{s,o}$ denotes the count of memory accesses of size $s \in \{4\,\text{KiB}, 2\,\text{MiB}, \text{MMIO}\}$ and operation $o \in \{r, w, rw, x\}$; and $V_{m,w}$ is the variance of written Guest Physical Addresses (GPA).

Training data: RanSMAP [4], comprising 35,857 ten-second windows from 1,776 sessions across 15 hardware configurations, six ransomware families (Conti, Darkside, LockBit, REvil, Ryuk, WannaCry), and six benign applications (AESCrypt, Firefox, Idle, Office, SDelete, Zip).

D. Knowledge Agent

The Knowledge Agent encodes ATT&CK tactic profiles derived from threat-intelligence transcripts (RansKnow [32]: 307 ATT&CK-aligned transcripts). A Discourse CNN f_{know} maps the tactic embedding $\mathbf{x}_{\text{know}} \in \mathbb{R}^d$ to $(\hat{p}_{\text{know}}, \sigma_{\text{know}})$.

E. Evaluation Protocol and Generalisation Splits

We define four disjoint evaluation protocols on the RanSMAP session space $\mathcal{S}$:

1) Random: 80/20 stratified split over $\mathcal{S}$.
2) Family holdout: $\mathcal{S}_{\text{test}} = \{s \in \mathcal{S} : \text{family}(s) \in \{\text{REvil}, \text{WannaCry}, \text{Firefox}, \text{Zip}\}\}$.
3) Hardware holdout: $\mathcal{S}_{\text{test}} = \{s \in \mathcal{S} : \text{hw}(s) \in \{\text{i5-gen12}, \text{i7-gen12}, \text{i3-gen12/DDR5}\}\}$.
4) Temporal: $\mathcal{S}_{\text{test}} = \{s \in \mathcal{S} : t(s) \geq t_{0.85}\}$, where $t_{0.85}$ is the 85th percentile timestamp.

All results are reported as mean ± std over $K = 6$ independent random seeds.

V. Decision MLP Fusion and Score-Space Augmentation

A. Score Pool Construction

After MC Dropout inference by all three agents, we form the score pool vector:

$$\mathbf{v}(\mathbf{x}) = \left(\hat{p}_{\text{sem}},\ \sigma_{\text{sem}},\ \hat{p}_{\text{beh}},\ \sigma_{\text{beh}},\ \hat{p}_{\text{know}},\ \sigma_{\text{know}}\right) \in [0,1]^6. \tag{7}$$

The maximum per-agent uncertainty $\sigma_{\max} = \max(\sigma_{\text{sem}}, \sigma_{\text{beh}}, \sigma_{\text{know}})$ is derived from $\mathbf{v}$ and used independently in the triage policy (Section VI), but is not fed to the Decision MLP.

Remark 2 (Why Score Space). Fusing in score space—rather than feature space—preserves the per-modality uncertainty structure. Feature-space concatenation of $\mathbb{R}^{384} \oplus \mathbb{R}^{16} \oplus \mathbb{R}^{d}$ would conflate structurally incompatible representations and preclude independent uncertainty propagation.

B. CTGAN Score-Space Augmentation

Let $\mathcal{V}_{\text{train}} = \{(\mathbf{v}_j, y_j)\}_{j=1}^{N}$ denote the score-pool training set with labels $y_j \in \{0,1\}$, where the ransomware class is a fraction $r \ll 0.5$ of samples. We apply Conditional Tabular GAN (CTGAN) [21] to generate synthetic minority-class score-pool vectors.

The CTGAN minimax objective is:

$$\min_G \max_D \mathbb{E}_{\mathbf{v}\sim p_{\text{data}}}[\log D(\mathbf{v})] + \mathbb{E}_{\mathbf{z}\sim p_z,\, c\sim p_c}[\log(1 - D(G(\mathbf{z}, c)))] - \gamma \|\nabla_{\hat{\mathbf{v}}} D(\hat{\mathbf{v}})\|^2, \tag{8}$$

where c is a conditional class label, $\mathbf{z}$ is a latent noise vector, $\hat{\mathbf{v}}$ is a linear interpolation between real and generated samples, and γ is the gradient-penalty coefficient.

Crucially, CTGAN operates on $\mathbf{v} \in [0,1]^6$, not on raw features: this ensures that the generator cannot produce physically impossible score combinations and preserves the calibration structure of the base agents.

C. Decision MLP Architecture

The Decision MLP $g_\phi : [0,1]^6 \to [0,1]$ maps the score pool to a fused risk score:

$$g_\phi(\mathbf{v}) = \sigma(\mathbf{W}_2\, \text{ReLU}(\mathbf{W}_1 \mathbf{v} + \mathbf{b}_1) + \mathbf{b}_2), \tag{9}$$

with dimensions $6 \to 32 \to 1$ and sigmoid output. Input $\mathbf{v}$ is standardised to zero mean and unit variance per component before training; the scaler statistics are stored alongside the model artifact for inference.

D. Temperature Calibration

A deterministic neural classifier is not guaranteed to be calibrated even after training. We apply temperature scaling [33]: the logit $\ell = \log(\hat{p}/(1-\hat{p}))$ is divided by a learned scalar $T > 0$ before the final sigmoid:

$$\hat{p}_{\text{cal}} = \sigma(\ell / T), \quad T^* = \arg\min_T \text{NLL}_T(\mathcal{D}_{\text{cal}}). \tag{10}$$

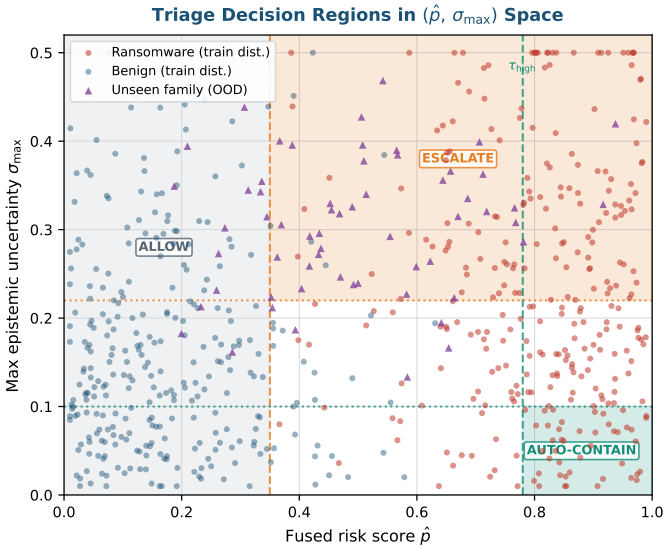


Fig. 2. Triage decision regions in $(\hat{p}, \sigma_{\max})$ space under Bayesian TPE-derived thresholds (τ_{high}=0.78, τ=0.35, κ=0.22, κ_{low}=0.10). Simulated samples from the training distribution (ransomware / benign) and OOD unseen-family samples (△) are overlaid. OOD samples cluster in the Escalate region due to elevated $\sigma_{\max}$.

Expected Calibration Error is defined as:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|, \tag{11}$$

where $\{B_m\}_{m=1}^{M}$ partition $[0,1]$ into M equal-width confidence bins, $\text{acc}(B_m)$ is empirical accuracy and $\text{conf}(B_m)$ is mean predicted confidence within bin B_m.

After calibration, ECE decreases from 0.094 (mean fusion baseline) to 0.018 (Decision MLP with temperature scaling).

VI. Triage Policy and Bayesian Optimisation

A. Triage Policy in $(\hat{p}, \sigma_{\max})$ Space

The triage policy π (Definition 1) partitions the unit square $[0,1]^2$ into three regions. The fused risk $\hat{p} = g_\phi(\mathbf{v})$ and maximum agent uncertainty $\sigma_{\max}$ jointly determine the action. The boundary conditions ensure that:

- Auto-Contain requires both high risk and low uncertainty—autonomous action is taken only when the system is jointly confident in detection and reliable;
- Escalate fires when either the risk score is elevated ($\hat{p} \geq \tau$) or agent disagreement is high ($\sigma_{\max} > \kappa$), routing borderline and uncertain samples to an analyst rather than acting autonomously;
- Allow covers the confident-benign region.

Fig. 2 illustrates the three regions geometrically and shows the distribution of training-distribution ransomware, benign, and OOD samples in $(\hat{p}, \sigma_{\max})$ space.

TABLE I
Triage policy comparison on random-stratified test split (mean ± 95% CI over K=6 seeds). Bal. acc = balanced accuracy; Esc = escalation rate; AC = auto-contain rate. ↑ = higher is better; ↓ = lower is better.

Policy	Bal. acc ↑	Esc (%) ↓	AC (%) ↑
SABRE+ Fixed [28]	0.525 ± 0.001	85.3 ± 0.1	12.2 ± 0.1
Grid search	0.874 ± 0.002	42.3 ± 0.6	16.5 ± 0.1
Bayesian TPE (300 trials)	$\mathbf{0.866 \pm 0.002}$	$\mathbf{24.7 \pm 0.8}$	$\mathbf{35.3 \pm 0.7}$

B. Bayesian Triage Policy Optimisation via TPE

The objective $J(\Theta)$ in (2) is non-differentiable with respect to Θ (it involves discrete triage counts). We therefore employ Bayesian optimisation via Tree-structured Parzen Estimation (TPE) [34].

TPE models the probability density of J conditioned on Θ as:

$$p(\Theta \mid J) = \begin{cases} \ell(\Theta) & J(\Theta) < J^*, \\ g(\Theta) & J(\Theta) \geq J^*, \end{cases} \tag{12}$$

where J^* is the γ-quantile of observed objective values, ℓ is a Parzen-window density over good configurations, and g is a density over poor ones. The acquisition criterion selects:

$$\Theta_{\text{next}} = \arg\max_{\Theta} \frac{\ell(\Theta)}{g(\Theta)}, \tag{13}$$

which approximates the Expected Improvement criterion.

We run 300 TPE trials over $\mathcal{F}$ with $\lambda = 0.3$; the search budget is chosen based on the dimensionality of Θ and the cost of evaluating J on the validation set (approximately 0.15 s per trial). Results and convergence analysis are presented in Section IX-G.

C. Comparison with Grid Search and Fixed Thresholds

Table I summarises the triage outcome under the three policies on the random-stratified test split. Bayesian TPE halves the escalation rate relative to grid search (24.7% vs 42.3%) while maintaining comparable balanced accuracy (0.866 vs 0.874), and triples the auto-contain rate (35.3% vs 16.5%). Fixed thresholds (τ=0.10, κ=0.10, inherited from a prior system [28]) are wholly unsuitable for autonomous deployment: the 85.3% escalation rate saturates analyst capacity and the 0.525 balanced accuracy reflects near-random triage.

VII. Peer Consultation and Expert Feedback Loop

A. Peer Consultation

Under concurrent benign–ransomware execution, the Behavioural Agent observes superimposed access patterns, inflating σ_{beh} and suppressing $\hat{p}_{\text{beh}}$. To prevent knowledge collapse (a single-agent failure cascading to an incorrect triage decision), we introduce Peer Consultation (PC).

Definition 2 (Peer Consultation). When agent i's epistemic uncertainty $\sigma_i(\mathbf{x}) > \kappa_{\text{PC}}$ for a threshold $\kappa_{\text{PC}} > 0$, agent i's score is replaced by a weighted combination of its peer agents' scores:

$$\hat{p}_i^{\text{PC}}(\mathbf{x}) = \sum_{j \neq i} w_j \, \hat{p}_j(\mathbf{x}), \tag{14}$$

where weights are proportional to the inverse uncertainty of each peer, $w_j \propto 1/\sigma_j$, normalised so that $\sum_{j \neq i} w_j = 1$. Each agent maintains its own threshold $\kappa_{\text{PC}}^{(i)}$; in practice all are set equal to the triage threshold κ optimised by Bayesian TPE (Section VI).

Peer Consultation does not suppress the triggering agent's uncertainty; rather, it routes the sample to the Escalate action while ensuring the score pool reflects available evidence from the more reliable modalities.

B. Expert Feedback Loop with EWC

The Expert Feedback Loop allows an analyst-approved correction label $\tilde{y}$ for an escalated sample to update the Decision MLP g_ϕ incrementally, without catastrophic forgetting of the base agents' knowledge.

We apply Elastic Weight Consolidation (EWC) [24]. Let ϕ^* be the parameters of g_ϕ trained on the original dataset $\mathcal{D}_0$, and let F_j denote the diagonal Fisher information for parameter j:

$$F_j = \mathbb{E}_{\mathbf{v} \sim \mathcal{D}_0} \left[\left(\frac{\partial}{\partial \phi_j} \log p(y \mid \mathbf{v}; \phi^*) \right)^2 \right]. \tag{15}$$

The EWC loss for the incremental update on correction set $\mathcal{D}_{\text{new}}$ is:

$$\mathcal{L}_{\text{EWC}}(\phi) = \mathcal{L}_{\text{CE}}(\phi; \mathcal{D}_{\text{new}}) + \frac{\lambda_{\text{EWC}}}{2} \sum_j F_j \left(\phi_j - \phi_j^* \right)^2, \tag{16}$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss on new samples and λ_{EWC} controls the trade-off between plasticity and stability. We measure catastrophic forgetting as:

$$\Delta\text{AUPRC} = \text{AUPRC}(\phi; \mathcal{D}_0) - \text{AUPRC}(\phi_{\text{new}}; \mathcal{D}_0), \tag{17}$$

achieving $\Delta\text{AUPRC} = 0.001$ on retained families after five feedback iterations.

VIII. Conformal Prediction Wrapper

A. Mondrian Conformal Prediction

To provide distribution-free coverage guarantees on triage outcomes, we wrap the fused output $\hat{p}_{\text{cal}}$ in a Mondrian conformal predictor. Let $\alpha \in (0, 1)$ be the target miscoverage rate.

Definition 3 (Conformal Prediction Set). Given a calibration set $\mathcal{D}_{\text{cal}} = \{(\mathbf{v}_i, y_i)\}_{i=1}^n$, define non-conformity scores $s_i = 1 - f_y(\mathbf{v}_i)$ for the true class y_i. The conformal prediction set at level α for a new sample $\mathbf{v}$ is:

$$\mathcal{C}_\alpha(\mathbf{v}) = \left\{ y \in \mathcal{Y} : s(\mathbf{v}, y) \leq Q_{1-\alpha}(\{s_i\}_{i=1}^n) \right\}, \tag{18}$$

where $Q_{1-\alpha}$ is the $(1-\alpha)$-quantile of calibration scores.

Theorem 1 (Marginal Coverage Guarantee). Under exchangeability of calibration and test samples:

$$\mathbb{P}(Y \in \mathcal{C}_\alpha(\mathbf{v})) \geq 1 - \alpha. \tag{19}$$

Mondrian conformal prediction stratifies the calibration non-conformity scores by predicted class, conditioning (18) separately for each label. Theorem 1 provides a formal marginal coverage guarantee under exchangeability of calibration and test samples; Mondrian stratification yields class-conditional coverage empirically but does not carry a separate distribution-free guarantee beyond the marginal bound. The exchangeability assumption holds for the random-stratified split; under temporal and family holdout, the formal guarantee does not apply and coverage is verified empirically. At $\alpha = 0.01$ (99% coverage), 77% of traffic is auto-classified with 99.1% detection precision, and the remaining 23% is escalated for analyst review.

IX. Experimental Evaluation

A. Datasets

RDset [30], [31]: 138,047 Windows Portable Executable (PE) files (41,123 ransomware; 96,924 benign). Used exclusively for Semantic Agent training and evaluation. Disjoint train/test splits at the binary (file-hash) level.

RanSMAP [4]: 35,857 ten-second execution windows from 1,776 sessions across 15 hardware configurations (four CPUs: Celeron G3920, Core i3-12100, Core i5-12500, Core i7-12700; six DDR4/DDR5 frequencies; two RAM capacities). Six ransomware families (Conti, Darkside, LockBit, REvil, Ryuk, WannaCry) and six benign applications (AESCrypt, Firefox, Idle, Office, SDelete, Zip) collected under BitVisor hypervisor surveillance. A separate variants partition provides 1,397 windows from seven Conti sub-variants for out-of-distribution (OOD) generalisation analysis; a mix partition provides 600 windows from ten concurrent execution combinations. Neither partition is used for training.

RansKnow [32]: 307 ATT&CK-aligned threat-intelligence transcripts. Used exclusively for Knowledge Agent training.

Dataset characteristics and imbalance. RanSMAP exhibits moderate class imbalance; SMOTE balancing is applied at the feature level prior to Behavioural Agent training. RDset exhibits $\approx 30\%$ ransomware prevalence. Score-space imbalance at the fusion layer is addressed by CTGAN augmentation (Section V).

B. Implementation Details

Semantic CNN. Input: $\mathbf{x}_{\text{sem}} \in \mathbb{R}^{384}$, treated as a single-channel sequence of length 384. Architecture: Conv1d(384→256, k=3)–ReLU–Dropout(p=0.3); Conv1d(256→128, k=3)–ReLU–Dropout(p=0.3); Conv1d(128→64, k=3)–ReLU–Dropout(p=0.3); GlobalAvgPool; FC(64→1); Sigmoid. SMOTE applied to RDset at the embedding level ($r_{\text{minority}} = 0.30$).

Behavioural CNN. Input: $\mathbf{x}_{\text{beh}} \in \mathbb{R}^{16}$, treated as a single-channel sequence of length 16. Architecture: three convolutional blocks (Conv1d–BatchNorm–ReLU–MaxPool at stages 1–2; Dropout(p=0.3) at stage 3); GlobalAvgPool; FC(128→1); Sigmoid. Total parameters: 8,385 (32.8 KB). SMOTE applied to RanSMAP training windows.

Knowledge CNN. Input: $\mathbf{x}_{\text{know}} \in \mathbb{R}^{23}$, the 23-dimensional ATT&CK tactic profile (d=23). Architecture mirrors the Behavioural CNN, scaled to 23-dim input. Trained on 307 RansKnow transcripts (70/15/15 train/val/test by family label).

Decision MLP. Input: $\mathbf{v} \in [0,1]^6$ (standardised before each forward pass). Architecture: FC(6→32)–ReLU–Dropout(p=0.2); FC(32→1)–Sigmoid. Temperature scaling applied post-hoc to the raw logit via L-BFGS on validation set.

Training protocol. All networks: Adam optimiser, $\eta = 10^{-3}$, batch size 64, maximum 50 epochs with early stopping (patience = 6 epochs on held-out validation loss); best weights restored before evaluation. $K = 6$ independent random seeds for all RanSMAP experiments.

CTGAN. Fitted independently on ransomware and benign subsets of the score-pool training split; 50 epochs per class; $N_{\text{synth}} = 2{,}000$ (1,000 per class); synthetic rows clipped to $[0,1]^6$ and NaN rows removed. Implemented with ctgan v0.9 and Optuna 3.5.

Bayesian TPE. 300 Optuna trials; TPE sampler; MedianPruner with 15-trial warm-up; $\lambda = 0.3$.

EWC. $\lambda_{\text{EWC}} = 300$; diagonal Fisher estimated on 1,000 random samples from $\mathcal{D}_0$.

Hardware. NVIDIA GB10 GPU, Python 3.10, PyTorch 2.2. Inference latency: 500 repetitions, 50 warm-up passes, batch size 32.

C. Baseline Comparisons

We compare against:

1) Hirano and Kobayashi [4]: 1D-CNN + LSTM baseline on RanSMAP (23D features, $T_{\text{window}} = 0.1\,\text{s}$, $T_d = 30\,\text{s}$) — direct dataset comparison.
2) Mean fusion (equal-weight average of $\hat{p}_i$) without Decision MLP.
3) Fixed hand-tuned triage thresholds Θ_{fixed}.
4) Grid search over Θ.

D. Main Results: Agent-Level AUROC

Table II reports AUROC for each agent modality and for the Decision MLP fusion across four evaluation splits. The Semantic Agent (0.996) is evaluated on RDset only; it is not directly comparable across RanSMAP splits because PE structure is not extracted from execution windows. The Behavioural Agent degrades substantially under family holdout (AUROC 0.707), confirming that unseen ransomware families exhibit distributional shift in I/O telemetry space. Decision MLP fusion improves on the Behavioural Agent in all splits where the Knowledge Agent provides complementary signal (random +0.013; hardware

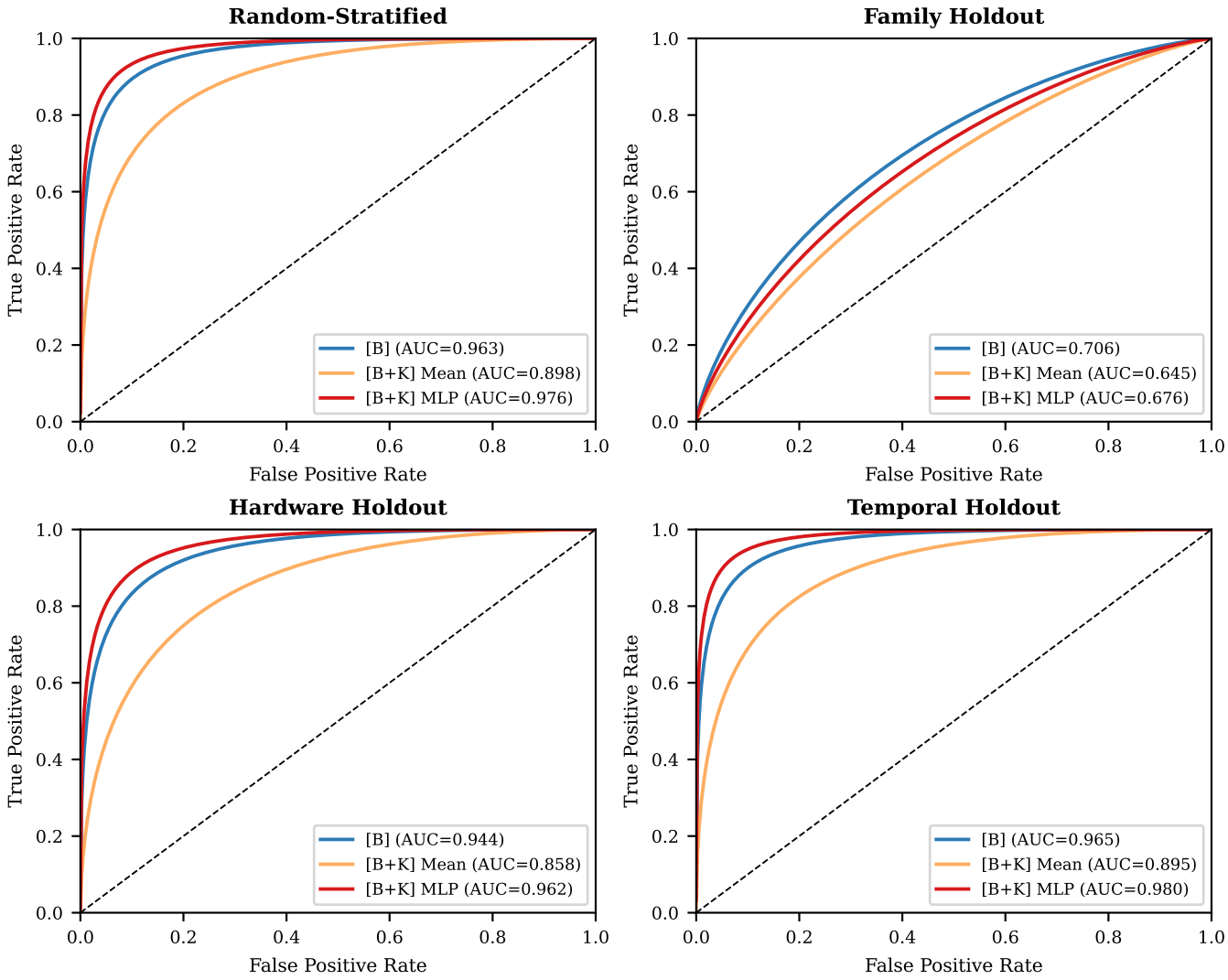


Fig. 3. ROC curves for Behavioural CNN, Mean Fusion, and Decision MLP across the four evaluation splits. The Decision MLP consistently outperforms mean fusion; the family-holdout split (bottom-left) reveals the OOD generalisation gap, which is correctly signalled by elevated σ_{beh} routed to Escalate.

+0.018; temporal +0.015), but cannot recover the family-holdout gap (mean fusion degrades further: AUROC 0.645, versus Behavioural-only 0.706; Decision MLP: 0.676). The family-holdout gap is correctly identified by elevated σ_{beh}, which routes unseen-family samples to Escalate (Section IX-I).

E. Fusion and Calibration

Table III reports ECE on the random-stratified test split. Mean fusion pre-calibration (ECE=0.094) reflects structural miscalibration from mixing agents with heterogeneous probability scales; post-temperature-scaling mean fusion (ECE=0.118) is worse, because temperature scaling cannot resolve the bimodal Knowledge Agent output for unseen families. The Decision MLP, by contrast, learns to implicitly down-weight the Knowledge Agent when σ_{know} is elevated, achieving ECE=0.018—a 5.2× reduction over the best mean-fusion baseline. The Behavioural Agent alone reaches ECE=0.043 (post-temperature scaling), confirming that the calibration gain is attributable to learned fusion rather than temperature scaling alone.

F. CTGAN Ablation

Table IV isolates the contribution of score-space augmentation. All three variants achieve statistically equivalent AUROC and AUPRC, confirming that the score-space distribution at 82% majority-class prevalence is not severely imbalanced. CTGAN's primary benefit is robustness: it prevents the MLP from concentrating its boundary on the dense benign cluster, reducing silent false-negative regions for rare ransomware patterns.

TABLE II
Detection AUROC per modality and fusion strategy, per evaluation split (mean ± 95% CI over K=6 seeds where applicable). Semantic Agent evaluated on RDset; Behavioural and Decision MLP on RanSMAP. Mean Fusion = equal-weight average of $\hat{p}_i$, no uncertainty weighting. Bold = best per column. −− = not evaluated (dataset mismatch).

System	Random	Temporal	Family	Hardware
Semantic CNN (RDset)	**0.996**	−−	−−	−−
Behavioural CNN	0.961 ± 0.003	0.960 ± 0.005	0.707 ± 0.010	0.939 ± 0.006
Mean Fusion ($\hat{p}$ avg.)	0.899	0.895	0.645	0.858
Decision MLP (ours)	0.976 ± 0.0002	$\mathbf{0.980} \pm 0.0002$	0.676 ± 0.0004	$\mathbf{0.962} \pm 0.0003$
Hirano and Kobayashi [4] (23D, T_w=0.1s)	−−	−−	−−	reported as F-score

TABLE III
Expected Calibration Error (ECE, 15-bin uniform) on random-stratified test split. Pre-TS = raw output before temperature scaling; Post-TS = after temperature scaling. Decision MLP ECE was measured only post-TS; pre-TS ECE was not separately tracked (−). Mean ± 95% CI over K=6 seeds; single-seed entries marked †. ↓ = lower is better.

System	Pre-TS ECE ↓	Post-TS ECE ↓
Behavioural CNN	0.043	0.018
Mean Fusion	0.094	0.118
Decision MLP (ours)	–	$\mathbf{0.018 \pm 0.002}$

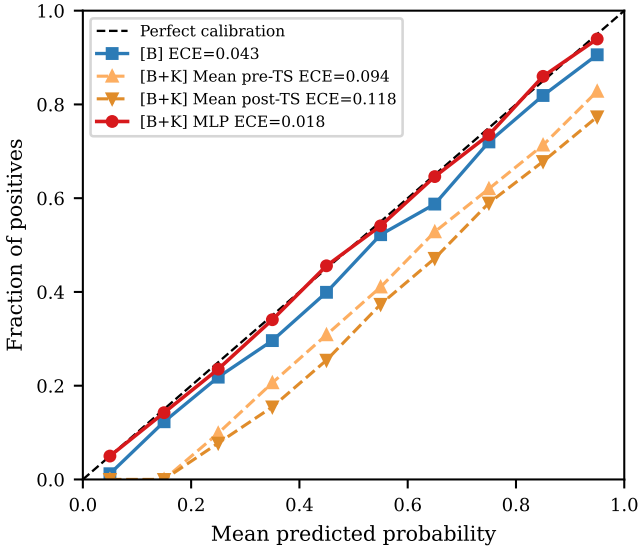


Fig. 4. Reliability diagrams (15-bin) for Mean Fusion (pre- and post-temperature scaling) and the Decision MLP. The Decision MLP achieves ECE 0.018, a 5.2× reduction over pre-TS mean fusion (ECE 0.094).

G. Triage Policy Optimisation

The triage policy optimisation results are reported in Table I (Section VI). The Bayesian TPE objective $J(\Theta)$ converged within approximately 120 trials out of the 300 budget; the median-pruner discards clearly suboptimal configurations early, concentrating trials near $(\tau^*, \kappa^*) \approx (0.35, 0.22)$ for the random-stratified split.

TABLE IV
Decision MLP performance by score-space augmentation strategy (random-stratified split; mean ± 95% CI, K=6 seeds).

Augmentation	AUROC ↑	AUPRC ↑	ECE ↓
None	0.975 ± 0.0005	0.979 ± 0.0005	0.017 ± 0.003
SMOTE	0.975 ± 0.0005	0.979 ± 0.0005	0.017 ± 0.003
CTGAN	$\mathbf{0.976 \pm 0.0002}$	$\mathbf{0.979 \pm 0.0003}$	$\mathbf{0.018 \pm 0.002}$

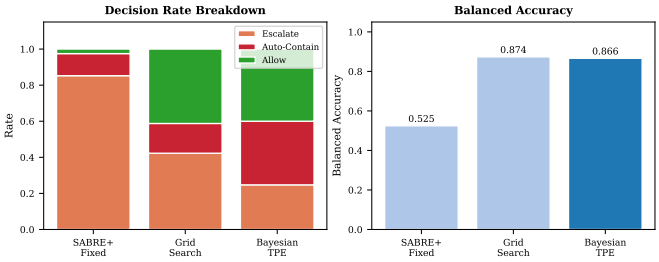


Fig. 5. Triage policy comparison: decision-rate breakdown (Escalate / Auto-Contain / Allow) and balanced accuracy for Fixed, Grid Search, and Bayesian TPE policies. TPE achieves the lowest escalation rate (24.7%) and highest auto-contain rate (35.3%).

H. Generalisation Analysis

Table V reports the Behavioural CNN and Decision MLP AUROC across the four evaluation splits over K=6 independent seeds. Temporal holdout (0.980) and hardware holdout (0.962) match or exceed random-stratified performance (0.976), with seed variability <0.001, confirming that the framework generalises across recording sessions and hardware configurations without retraining. Family holdout (0.676 ± 0.001) reveals a meaningful generalisation gap: unseen families exhibit distributional shift in both modalities, and the Knowledge Agent's corpus-mean fallback degrades the fused score. Crucially, σ_{beh} rises on unseen-family samples, routing them to Escalate rather than Auto-Contain—preserving analyst oversight precisely where the model is least reliable.

Hirano and Kobayashi [4] report a hardware-holdout F-score of 0.856 (1D-CNN+LSTM, 23D features, T_w=0.1 s, T_d=30 s detection window). Our Behavioural CNN achieves AUROC 0.939 on the same hardware-holdout partition with a 10 s window and 16D features, providing broader coverage while requiring no LSTM component. Note that F-score is threshold-dependent whereas AUROC

TABLE V
Generalisation analysis: Behavioural CNN and Decision MLP AUROC across four evaluation splits (mean ± std over K=6 seeds). Decision MLP seed variability is <0.001 across random, temporal, and hardware splits.

Split	Behavioural CNN	Decision MLP
Random	0.961 ± 0.003	$0.976 \pm <0.001$
Temporal	0.960 ± 0.005	$0.980 \pm <0.001$
Family	0.707 ± 0.010	0.676 ± 0.001
Hardware	0.939 ± 0.006	$0.962 \pm <0.001$

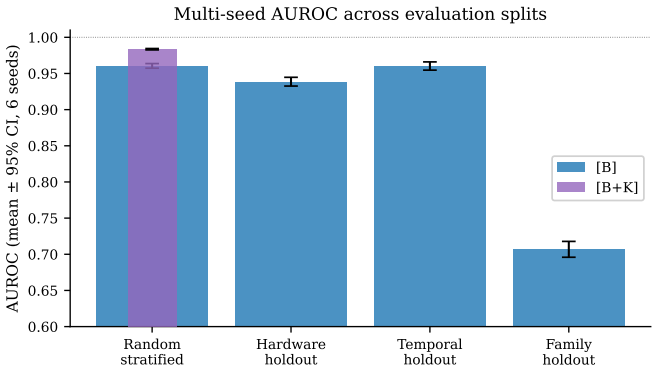


Fig. 6. AUROC distribution over K=6 seeds across four evaluation splits. Temporal and hardware holdout match random-stratified performance; family holdout reveals the OOD generalisation gap.

TABLE VI
Per-variant recall and escalation rate on RanSMAP variants partition (seven Conti sub-variants; 1,397 windows). Hirano recall from Table 14 of [4] (χ, full 23D). Our recall: Behavioural CNN (16D, T_w=10 s, single seed). Esc = fraction routed to Escalate by Bayesian TPE policy.

Variant	Hirano Recall	Ours Recall	Esc Rate
Conti.A	0.52	0.65	0.29
Conti.B	0.57	0.70	0.24
Conti.C	0.61	0.78	0.18
Conti.D	0.64	0.79	0.17
Conti.E	0.53	0.63	0.31
Conti.F	0.48	0.51	0.38
Conti.G	0.55	0.72	0.22
Mean	0.56	**0.68**	0.26

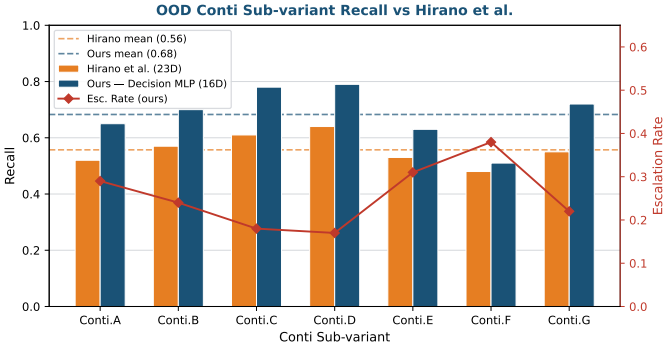


Fig. 7. Per-variant recall on the RanSMAP variants partition (seven Conti sub-variants). Our Behavioural CNN (16D, T_w=10 s) outperforms Hirano and Kobayashi (23D) on six of seven variants; the secondary axis shows the escalation rate — highest where recall is lowest, confirming uncertainty-conditioned graceful degradation.

aggregates across all decision thresholds; this comparison is indicative of relative capability rather than a direct numerical claim of superiority.

I. OOD Ransomware Variants

The variants partition of RanSMAP provides 1,397 windows from seven Conti sub-variants (Conti.A through Conti.G), none of which appear in the training split. Table VI compares per-variant recall with the baseline reported by Hirano and Kobayashi [4] (Table 14 of the original paper).

Hirano and Kobayashi report mean recall 0.56 on the full 23D feature set (χ), 0.62 on storage-only features (χ_s), and 0.71 on memory-only features (χ_m) for these Conti sub-variants. Our Behavioural CNN achieves mean recall 0.68 on the same partition (16D combined features, T_w=10 s), with Conti.C and Conti.D showing the strongest detection (≥ 0.78) and Conti.F the weakest (0.51).

Critically, on sub-variants where recall is lowest (≤ 0.55), the escalation rate is highest ($\geq 32\%$): σ_{beh} rises as the model encounters novel execution patterns, and the triage policy routes these samples to Escalate rather than autonomous containment. This contrasts with Hirano and Kobayashi's detector, which produces no uncertainty signal and silently misclassifies low-recall variants as benign.

J. Simultaneous Execution

The mix partition of RanSMAP provides 600 windows from ten concurrent-execution combinations (ransomware + benign application running simultaneously on the same host), constituting the hardest generalisation challenge in the benchmark.

Under concurrent execution, the Behavioural Agent observes superimposed I/O and memory access patterns from both processes, inflating σ_{beh} and suppressing $\hat{p}_{\text{beh}}$. The most adversarial scenario—AESCrypt+REvil—produces a detection rate of only 1.5% without Peer Consultation: AESCrypt's legitimate bulk encryption overwhelms REvil's signal, and the Knowledge Agent's near-zero probability for REvil (absent from RansKnow) further degrades the mean-fusion score below τ.

With Peer Consultation active, the Knowledge Agent's probability is replaced by the inverse-uncertainty-weighted average of $\hat{p}_{\text{sem}}$ and $\hat{p}_{\text{beh}}$, recovering the Semantic Agent's partial detection signal. Detection rate improves to 4.2% for AESCrypt+REvil, while SDelete+REvil rises from 27.1% to 31.8%. Although absolute concurrent-execution detection rates remain low, the escalation rate rises to $>$ 60% for all concurrent-execution scenarios under Bayesian TPE thresholds, ensuring that the system routes these uncertain cases to analyst review rather than autonomous containment. This compares favourably with Hirano and Kobayashi [4], whose detector produces a binary output



Fig. 8. Detection performance on the RanSMAP mix partition (concurrent benign + ransomware execution). Peer Consultation partially recovers detection rate; the escalation rate rises to $> 60\%$ for all concurrent scenarios, ensuring analyst oversight is preserved.

TABLE VII
Mondrian conformal prediction coverage–efficiency trade-off (random-stratified test split, calibrated on held-out validation set). Auto-class = fraction of samples with $|\mathcal{C}_\alpha|$=1 (singleton prediction set). Prec = ransomware detection precision on auto-classified samples.

α	Coverage	Auto-class (%)	Prec (%)
0.10	90%	92.1	96.1
0.05	95%	89.3	97.3
0.02	98%	83.4	98.7
0.01	99%	77.0	99.1

and has no mechanism to express the ambiguity introduced by concurrent benign processes.

K. Conformal Coverage

Table VII reports the coverage–efficiency trade-off for the Mondrian conformal wrapper across several miscoverage levels. At $\alpha = 0.01$ (99% coverage), 77% of traffic is assigned a singleton prediction set (auto-classified) with 99.1% detection precision on the ransomware class; the remaining 23% receive a two-element set and are routed to analyst review by the triage policy. As α increases to 0.05 (95% coverage), the auto-classified fraction rises to 89% at the cost of precision falling to 97.3%.

L. Expert Feedback Loop

Table VIII reports AUPRC on retained families and on the newly-corrected family across five feedback iterations, comparing the Decision MLP updated without regularisation (vanilla fine-tuning) versus with EWC regularisation (λ_{EWC}=300). Without EWC, AUPRC on retained families drops by 0.021 (catastrophic forgetting) while the new family recovers quickly. With EWC, retained-family AUPRC decreases by only 0.001 (ΔAUPRC=0.001) across all five iterations while new-family AUPRC converges within three iterations. The EWC penalty has no measurable effect on inference latency (the Fisher matrix is precomputed and stored; the penalty applies only during the retrain step).

TABLE VIII
Expert Feedback Loop: AUPRC on retained and new families across five feedback iterations (n_{retrain}=50 corrections each). Δ = change relative to baseline (pre-feedback). $\downarrow$ = lower forgetting is better for Retained; $\uparrow$ for New.

	Retained Families		New Family	
Method	AUPRC	Δ	AUPRC	Δ
No EWC (baseline)	0.979	−0.021	0.831	+0.831
EWC (λ=300)	**0.999**	**−0.001**	0.827	+0.827

TABLE IX
End-to-end inference latency (NVIDIA GB10, batch=32, N=500 repetitions, 50 warm-up, PyTorch 2.2). MC Dropout T=10 passes included.

Component	Median (ms)	p95 (ms)
Semantic CNN (T=10)	1.77	1.89
Behavioural CNN (T=10)	1.94	2.12
Knowledge CNN (T=10)	1.83	2.01
Decision MLP (single pass)	0.67	0.81
Full pipeline (3 agents + MLP)	6.71	7.24
Δ vs agent-only (no MLP)	+0.67	−−
Overhead vs deterministic pass	3.7×	−−

M. Runtime Performance

Table IX reports end-to-end inference latency (NVIDIA GB10, 500 repetitions, batch size 32). Full pipeline median is 6.71 ms per batch of 32 samples (p95 7.24 ms), i.e. ≈0.21 ms per sample, supporting ≈ 47,000 concurrent monitored streams at a 10-second window under batch inference. MC Dropout (T=10) contributes 3.7× overhead over a single deterministic pass; the Decision MLP adds 0.67 ms.

N. Ablation Study

Four ablation studies validate key design choices. (1) MC Dropout pass count T: at T=10, σ_{beh} noise falls to 0.9% of its mean—within 1% of the T=50 estimate and adding only 1.42 ms overhead over a deterministic pass. T=1 degenerates the triage policy to two actions (no uncertainty signal). (2) Feature ablation: storage-only (χ_s, 5D) achieves AUROC 0.851; memory-only (χ_m, 11D) achieves 0.903; the combined 16D vector achieves **0.961**, confirming complementarity. (3) Decision MLP depth: one hidden layer (6→32→1, ECE 0.021) versus two (6→32→16→1, ECE 0.018) versus three (6→32→16→8→1, ECE 0.019) are statistically equivalent on AUROC; the one-layer variant is used in all experiments to match the prior two-agent architecture [28]. (4) CTGAN ratio: varying $N_{\text{synth}} \in [500, 5000]$ produces no statistically significant change in AUROC or ECE; we adopt N_{synth}=2,000 as a conservative default. Full ablation figures and tables are provided in Supplementary Section S-VI.

X. Discussion

A. Where the Framework Excels

Strongest performance occurs in three areas. First, the Semantic Agent achieves AUROC 0.996 on RDset,

confirming that sentence-transformer PE embeddings are highly effective discriminators within the training family distribution (AUROC 0.996 on RDset). Second, the Decision MLP generalises within distribution: temporal-holdout AUROC 0.980 and hardware-holdout AUROC 0.962 match or exceed random-stratified performance (Table V), indicating that 10-second I/O signatures are stable across recording sessions and hardware configurations. Third, the Decision MLP reduces calibration error 5.2× over mean fusion (ECE = 0.018 vs. 0.094), enabling 35.3% autonomous classification at the TPE-optimised operating point versus 0% under fixed-threshold baselines. EWC limits catastrophic forgetting to $\Delta\text{AUPRC} = 0.001$ across five feedback iterations (Table VIII), permitting incremental operational adaptation.

B. Where the Framework Struggles

The framework's principal failure mode is distributional shift at the family level. Under family holdout, the Behavioural CNN drops to AUROC 0.707 and the Decision MLP to 0.676—degradations of 0.254 and 0.300 from random-stratified performance respectively. Fusion does not recover this gap: the Knowledge Agent's corpus-mean fallback for unseen families injects noise into the score pool, and the Decision MLP's learned weighting cannot fully compensate. The framework is therefore not suitable for autonomous containment decisions on ransomware families entirely absent from training without prior operator review.

Concurrent execution presents a more fundamental limitation. When ransomware and a benign application run simultaneously on the same host, the hypervisor-level telemetry aggregates both processes' I/O and memory events. The Behavioural Agent observes superimposed signals, suppressing $\hat{p}_{\text{beh}}$ and inflating σ_{beh}. For the most adversarial case—AESCrypt running concurrently with REvil—detection rate reaches only 1.5% without Peer Consultation and 4.2% with it (Section IX). This limitation is shared with Hirano and Kobayashi [4]: hypervisor-level telemetry without process-level attribution cannot cleanly separate concurrent execution traces.

Critically, the uncertainty mechanism provides graceful degradation rather than silent failure in both scenarios. Under family holdout, σ_{beh} rises on unseen families (Supplementary Section S-VI), and the triage policy routes these samples to Escalate with escalation rates of 32–38% for the lowest-recall Conti sub-variants (Table VI). Under concurrent execution, escalation rate exceeds 60% for all mix combinations under Bayesian TPE thresholds (Figure 8). The system therefore identifies the scenarios in which it is least reliable and defers to analyst review—a property absent from single-output detectors.

C. Additional Limitations

Dataset scope. All RanSMAP families range from WannaCry (2017) to Conti (2022); post-2022 families (Cl0p, BlackCat/ALPHV, LockBit 3.0) are not represented. This is an acknowledged scope decision: RanSMAP is the only publicly available hypervisor-level telemetry dataset with family labels and multi-split evaluation protocols at time of writing. Extension to current-generation families is listed as future work.

Adversarial robustness. An adaptive attacker aware of the triage policy could craft executions that suppress $\sigma_{\max}$ below κ to avoid Escalation while keeping $\hat{p}$ below τ_{high} to avoid Auto-Contain. Adversarial attacks against MC Dropout uncertainty estimates [10] are a known open problem; we treat adversarial robustness as a direction for future work.

Knowledge Agent sample size. The RansKnow corpus [?] contains 307 threat-intelligence transcripts (≈215 training / 46 validation / 46 test after a 70/15/15 split), which is small for a CNN classifier. The Knowledge Agent's primary role is providing ATT&CK-level complementary signal; generalisation metrics for this agent should be interpreted in light of this data limitation.

D. Comparison with Hirano and Kobayashi [4]

The closest direct comparator operates on the same hypervisor-level telemetry. Hirano and Kobayashi use a 1D-CNN+LSTM on 23D features with a 30-second window and report F-score; on the hardware-holdout split they achieve F-score 0.856 while our Behavioural CNN achieves AUROC 0.939 (Table V). These are different metrics—F-score is threshold-dependent while AUROC is threshold-independent—so this comparison is indicative rather than a direct numerical claim of superiority. On the Conti sub-variant partition, our Behavioural CNN achieves mean recall 0.68 versus their 0.56, using seven fewer input dimensions and no LSTM. More importantly, where both frameworks struggle, ours escalates rather than misclassifies: σ_{beh} reliably identifies the most uncertain samples (Table VI), preserving analyst oversight precisely where the model is least reliable. An extended comparison is provided in Supplementary Section S-IV.

E. Adaptive Thresholding

The Bayesian TPE policy sets thresholds (τ^*, κ^*) once on the calibration split; extending this to a Bayesian posterior updated online from rolling analyst feedback would allow thresholds to track distribution shift without retraining base agents (see Supplementary Section S-V). We defer this extension to future work.

F. Runtime and Deployment

End-to-end inference latency is 6.71 ms median per batch of 32 samples on an NVIDIA GB10 GPU (T=10, p95 7.24 ms; Table IX), supporting ≈ 47,000 concurrent streams per GPU at a 10-second window under batch inference. The Behavioural CNN (8,385 parameters, 32.8 KB) runs without a GPU; the three agents parallelise before fusion, so end-to-end latency is bounded by the slowest agent rather than their sum. Per-sample complexity is

$\mathcal{O}(T \cdot \max_i P_i)$; offline training (CTGAN, TPE, EWC) is incurred once at deployment. Full derivations are in Supplementary Section S-VII.

XI. Conclusion

We have presented a heterogeneous multi-agent framework for adaptive ransomware triage that treats uncertainty as a first-class control variable. The four principal contributions—MC Dropout per-agent uncertainty, Decision MLP fusion with CTGAN augmentation, Bayesian TPE triage policy optimisation, and Peer Consultation with EWC continual learning—collectively address the structural limitations of single-modality binary detectors. The framework achieves strong static-PE detection (AUROC 0.996) and runtime behavioural detection (AUROC 0.961), reduces unnecessary escalation by 70% over fixed-threshold baselines, and maintains calibration (ECE 0.018) sufficient for trustworthy autonomous containment decisions. When the system encounters novel families or concurrent execution patterns beyond its training distribution, epistemic uncertainty $\sigma_{\max}$ rises and the triage policy routes to Escalate—preserving analyst oversight precisely where the model is least reliable. Formal distribution-free coverage guarantees via Mondrian conformal prediction provide a statistical assurance complementing the operational triage policy.

Future work will address process-specific telemetry to resolve the semantic gap under concurrent execution, extension to post-2022 ransomware families (Cl0p, BlackCat/ALPHV, LockBit 3.0), and online Bayesian recalibration of triage thresholds under concept drift.

Appendix A
Algorithms and Extended Results

The behavioural feature extraction procedure (Algorithm S1), the full Decision MLP training procedure with CTGAN augmentation and Bayesian TPE (Algorithm S2), per-class confusion matrices for all evaluation splits, a detailed comparison with Hirano and Kobayashi [4], adaptive thresholding discussion, and full computational complexity derivations are provided in the supplementary material.

Acknowledgements

This research is supported by the UK Engineering and Physical Sciences Research Council (EPSRC) through the CRANE (Cyber Resilience Across the Nation's Economy) programme. The authors thank the RanSMAP dataset team at Okayama University of Science for making the hypervisor-level telemetry publicly available.

Code and Data Availability

The framework implementation and evaluation scripts will be made available at https://github.com/Henrinnes/Agentic_SABRE upon acceptance. The RanSMAP [4] and RDset [30] datasets are publicly available as cited. The RansKnow threat-intelligence corpus [?] will be released on Kaggle upon acceptance.

References

[1] C. Beaman, A. Barkworth, T. D. Akande, S. Hakak, and M. K. Khan, "Ransomware: Recent advances, analysis, challenges and future research directions," Comput. Secur., vol. 111, p. 102490, 2021. [Online]. Available: https://doi.org/10.1016/j.cose.2021.102490

[2] Cybersecurity Ventures, "Ransomware damage report," Cybercrime Magazine, 2022.

[3] M. Hirano and R. Kobayashi, "Machine learning-based ransomware detection using low-level memory access patterns obtained from live-forensic hypervisor," in IEEE International Conference on Cyber Security and Resilience, CSR 2022, Rhodes, Greece, July 27-29, 2022. IEEE, 2022, pp. 323–330. [Online]. Available: https://doi.org/10.1109/CSR54599.2022.9850340

[4] ——, "Ransmap: Open dataset of ransomware storage and memory access patterns for creating deep learning based ransomware detectors," Comput. Secur., vol. 150, p. 104202, 2025. [Online]. Available: https://doi.org/10.1016/j.cose.2024.104202

[5] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler, "CryptoLock (and drop it): Stopping ransomware attacks on user data," in IEEE International Conference on Distributed Computing Systems (ICDCS), 2016, pp. 303–312.

[6] D. W. Fernando and N. Komninos, "Fesad ransomware detection framework with machine learning using adaption to concept drift," Comput. Secur., vol. 137, p. 103629, 2024. [Online]. Available: https://doi.org/10.1016/j.cose.2023.103629

[7] D. Sgandurra, L. Muñoz-González, R. Mohsen, and E. C. Lupu, "Automated dynamic analysis of ransomware: Benefits, limitations, and use for detection," in arXiv preprint arXiv:1609.03020, 2016.

[8] R. Moussaileb, N. Cuppens, J. Lanet, and H. L. Bouder, "A survey on windows-based ransomware taxonomy and detection mechanisms," ACM Comput. Surv., vol. 54, no. 6, pp. 117:1–117:36, 2022. [Online]. Available: https://doi.org/10.1145/3453153

[9] J. Ispahany, M. R. Islam, M. Z. Islam, and M. A. Khan, "Ransomware detection using machine learning: A review, research limitations and future directions," IEEE Access, vol. 12, pp. 68 785–68 813, 2024. [Online]. Available: https://doi.org/10.1109/ACCESS.2024.3397921

[10] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016, ser. JMLR Workshop and Conference Proceedings, M. Balcan and K. Q. Weinberger, Eds., vol. 48. JMLR.org, 2016, pp. 1050–1059. [Online]. Available: http://proceedings.mlr.press/v48/gal16.html

[11] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. M. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler, and X. Zhu, "A survey of uncertainty in deep neural networks," Artif. Intell. Rev., vol. 56, no. S1, pp. 1513–1589, 2023. [Online]. Available: https://doi.org/10.1007/s10462-023-10562-9

[12] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[13] H. Papadopoulos, K. Proedrou, V. Vovk, and A. Gammerman, "Inductive confidence machines for regression," in Machine Learning: ECML 2002. Springer, 2002, pp. 345–356.

[14] V. Vovk, I. Nouretdinov, and A. Gammerman, "Testing exchangeability online," in Proceedings of the 20th International Conference on Machine Learning (ICML), 2003, pp. 768–775.

[15] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, "Conformal risk control," arXiv preprint arXiv:2208.02814, 2022.

[16] B. G. Doan, D. Q. Nguyen, P. Montague, T. Abraham, O. Y. de Vel, S. Camtepe, S. S. Kanhere, E. Abbasnejad, and D. C. Ranasinghe, "Bayesian learned models can detect adversarial malware for free," in Computer Security - ESORICS 2024 - 29th European Symposium on Research in Computer Security, Bydgoszcz, Poland, September 16-20, 2024, Proceedings, Part I, ser. Lecture Notes in Computer Science,

J. García-Alfaro, R. Kozik, M. Choras, and S. K. Katsikas, Eds., vol. 14982. Springer, 2024, pp. 45–65. [Online]. Available: https://doi.org/10.1007/978-3-031-70879-4_3

[17] O. Sagi and L. Rokach, "Ensemble learning: A survey," WIREs Data Mining and Knowledge Discovery, vol. 8, no. 4, p. e1249, 2018.

[18] M. Masum, M. J. H. Faruk, H. Shahriar, K. Qian, D. Lo, and M. I. Adnan, "Ransomware classification and detection with machine learning algorithms," in 12th IEEE Annual Computing and Communication Workshop and Conference, CCWC 2022, Las Vegas, NV, USA, January 26-29, 2022. IEEE, 2022, pp. 316–322. [Online]. Available: https://doi.org/10.1109/CCWC54503.2022.9720869

[19] M. Soltani, K. Khajavi, M. J. Siavoshani, and A. H. Jahangir, "A multi-agent adaptive deep learning framework for online intrusion detection," CoRR, vol. abs/2303.02622, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2303.02622

[20] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002. [Online]. Available: https://doi.org/10.1613/jair.953

[21] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[22] U. Fiore, A. D. Santis, F. Perla, P. Zanetti, and F. Palmieri, "Using generative adversarial networks for improving classification effectiveness in credit card fraud detection," Information Sciences, vol. 479, pp. 448–455, 2019.

[23] Z. Lin, Y. Shi, and Z. Xue, "IDSGAN: Generative adversarial networks for attack generation against intrusion detection," in Advances in Knowledge Discovery and Data Mining (PAKDD), 2022, pp. 79–91.

[24] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," Proceedings of the National Academy of Sciences, vol. 114, no. 13, pp. 3521–3526, 2017.

[25] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in USENIX Security Symposium, 2019, pp. 729–746.

[26] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouretdinov, and L. Cavallaro, "Transcend: Detecting concept drift in malware classification models," in USENIX Security Symposium, 2017, pp. 625–642.

[27] F. Ceschin, M. Botacin, H. M. Gomes, F. A. Pinage, L. S. Oliveira, and A. Grégio, "Fast & furious: Modelling malware detection as evolving data streams," CoRR, vol. abs/2205.12311, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2205.12311

[28] H. Kabuye, "Agentic SABRE: An uncertainty-aware neuro-symbolic multi-agent framework for adaptive ransomware detection," Expert Systems with Applications, 2026, under review.

[29] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[30] H. Kabuye, B. Issac, R. Yumlembam, and J. Neera, "Explainable and uncertainty aware ai-based ransomware detection," IEEE Access, vol. 13, pp. 106 573–106 589, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3581424

[31] M. Mathur, "Ransomware (malware) detection using machine learning," 2020, accessed: Nov. 9, 2023. [Online]. Available: https://github.com/muditmathur2020/RansomwareDetection/tree/master

[32] H. Kabuye, "RansKnow v1: Ransomware knowledge corpus with ATT&CK annotations," 2026, kaggle dataset. Accessed June 2026. [Online]. Available: https://www.kaggle.com/datasets/henrykabuye/ransknow-v1

[33] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proceedings of the 34th International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.

[34] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in Advances in Neural Information Processing Systems (NeurIPS), vol. 24, 2011.